

API SPECIFICATION

MapMob Admin Dashboard

Every endpoint the admin dashboard needs, with parameters, request bodies and response shapes — and a gap analysis against the API that exists today.

Prepared for	MapMob backend team
Date	27 September 2026
Frontend	Angular 21 admin dashboard
API tested	https://api.mapmob.com.co
Endpoints specified	135

5

Ready to use

26

Exist, need changes

104

Not built yet

The dashboard is fully built and runs today against local fixtures. Nothing on the frontend is waiting to be designed — every screen, form and table already exists and already knows the exact shape of data it needs. That shape is what this document writes down. Section 2 lists the four problems that have to be solved before any screen can talk to the live API.

1. Conventions

What this document covers

This is the API for the **Angular admin dashboard** only. It was written by reading every HTTP call the dashboard makes (132 calls across 17 features) and every model it deserialises into, then checking each one against the live API.

The Postman collection `Map Mop.postman_collection.json` holds **55 saved requests: 37 under Admin and 18 under Api**. All 37 admin requests are accounted for in this document. The 18 under Api are the mobile app's own endpoints — registration, OTP, forgot/reset password, profile, saved location, home feed and the user-facing report form. **The dashboard calls none of them**, so they are out of scope here and nothing in this document asks you to change them.

Three endpoints in section 3 are marked *Not built* even though they are not yet called by the dashboard either — the place create, update and form-options endpoints. They are included because the place form is built and needs them; see issue 3.

	COUNT	NOTE
Admin requests in the collection	37	All mapped in this document.
Api (mobile) requests in the collection	18	Out of scope. The dashboard calls none.
HTTP calls the dashboard actually makes	132	Every one is specified in section 3.
Endpoints specified here	135	The 132 above, plus 3 the place form needs.

Base URL

The dashboard is configured with a single base URL and calls every path in section 3 beneath it. Today it points at `http://localhost:3000/api` in `.env`; in production it will point at the live API.

Every path in this document is written **relative to that base URL**. So `/users` means `<base>/users`. The existing admin API serves its routes under `/admin`, so if the base URL becomes `https://api.mapmob.com.co/admin` then `/users` resolves to `https://api.mapmob.com.co/admin/users`, which already works. Keeping all admin routes under one prefix is what makes that possible — please do not split them across `/admin` and `/api`.

Authentication

Laravel Sanctum bearer tokens, as the API already does. The dashboard signs in once, stores the token, and must send it on every later call as `Authorization: Bearer <token>`.

Expected failure behaviour: `401` when the token is missing, invalid or expired, and `403` when the admin is authenticated but the role lacks the permission. The dashboard has a roles-and-permissions feature, so `403` needs to be distinct from `401` — it signs the admin out on `401` but not on `403`.

Response envelope

Please return the payload directly, with no wrapper. The dashboard deserialises the response body straight into the models in this document. The current API uses three different envelopes across different resources, which the dashboard cannot read:

ENVELOPE SEEN TODAY	STYLE	USED BY	PROBLEM
<code>{data, links, meta}</code>	Laravel collection	categories, stores, regions, cities	Third shape
<code>{status_code, message, data}</code>	custom	users, ads, reports	No pagination total
<code>{status_code, message, data, meta}</code>	custom	offers, products	Second shape

A list endpoint must return exactly this:

```
{
  "items": [ { ... }, { ... } ],
  "totalCount": 137
}
```

A single-object endpoint must return the object itself:

```
{ "id": "12", "name": "...", "status": "active" }
```

`totalCount` is the count across ALL pages, not the length of `items`. The dashboard needs it to draw the pager. An envelope without a total (the `{status\_code, message, data}` shape) cannot drive a paged table.

Pagination

The dashboard sends `pageIndex` and `pageSize` on every list endpoint. **pageIndex is zero-based** — the first page is `pageIndex=0`. Laravel's `page` is one-based, so the API must either accept `pageIndex` as its own parameter or translate `page = pageIndex + 1` internally.

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based. First page is 0.
pageSize	integer	yes	Rows per page. The dashboard sends 4-20 depending on the screen.
search	string	no	Free-text search. Omitted when empty.
sort	string	no	One of: newest, oldest, name. Omitted when not chosen.

Field naming and types

camelCase, not snake_case. The dashboard reads `isActive`, `createdAt`, `parentId`, `placeCount`. The API currently returns `is_active`, `created_at`, `parent_id`. Every field name in section 3 is the exact name expected.

IDs are strings. The dashboard types every id as a string, so `"12"` rather than `12`. Numeric ids in JSON work too, but strings are what the models declare, so please return strings.

Dates are ISO 8601 strings (`2026-09-27T18:00:00Z`), or `YYYY-MM-DD` where the field is a plain date such as `startsOn`. The current API accepts `22-9-2026` on writes, which is ambiguous — please accept and return ISO dates.

Text is a single resolved string, not a translation object. The dashboard is Arabic-only and reads `"name": "مطاعم"`. The API returns `{"ar": ..., "en": ...}` on some endpoints. Please resolve the language server-side from the `Accept-Language` header (or default to Arabic) and return a plain string. Write endpoints may keep taking `name[ar]` / `name[en]`.

Status is a word, not a flag. The dashboard reads `"status": "active"` / `"suspended"`, not `is_active: true`. The allowed values per resource are given with each endpoint.

Method override

The dashboard sends real `PUT`, `PATCH` and `DELETE` requests. It does **not** use the `_method` field that the current Postman collection relies on. Please route the real verbs.

`PUT` replaces the whole record. `PATCH` changes one thing — it is used only for the small status toggles, and its body carries just that field.

File uploads

Endpoints that carry a file use `multipart/form-data` ; everything else uses `application/json` . The content type is stated on each endpoint below.

Upload endpoints also send a matching `isXRemoved` boolean (for example `isImageRemoved`). When it is `true` the existing file should be deleted and the field set to null, even though no new file was sent. When it is `false` and no file is sent, keep the current file.

Responses return the stored file as an absolute URL (for example `imageUrl` , `logoUrl`), which the current API already does correctly.

Errors

Validation failures should return `422` with the field errors, which the API already does correctly on all create endpoints. Unknown ids should return `404` . Please never return `500` for bad input — see issue 4.

422 Unprocessable Entity

```
{
  "message": "The name field is required.",
  "errors": { "name": ["The name field is required."] }
}
```

2. Blocking issues

These stop the dashboard from working against the live API at all. They come before the endpoint list.

1 The dashboard never sends the auth token

BLOCKER

This one is on the frontend, and it is listed first because it blocks everything else. The dashboard signs in, receives the token and saves it to `localStorage`, but there is no HTTP interceptor, so no later request carries an `Authorization` header. Every authenticated call would come back `401`.

No backend work is needed here — it is recorded so both sides know it is being handled, and so the backend team is not asked to debug a wave of `401`s that are the frontend's fault.

What to do: Frontend task: add an Angular HTTP interceptor that attaches the stored bearer token to every outgoing request, and signs the admin out on a `401` response.

2 Three quarters of the endpoints do not exist

BLOCKER

The dashboard needs 135 endpoints. 31 exist in some form. 104 return `404` today, including every endpoint for eight whole features: **dashboard, reviews, subscriptions, payments, analytics reports, notifications, inbox, content pages and settings**.

Those features are fully built on the frontend — tables, forms, detail pages, filters, exports. They currently read local fixtures. They cannot be switched on at all until the endpoints exist. Section 4 lists exactly which.

What to do: Build the endpoints in section 3. The suggested order is in section 4: the shared conventions first, then places and users (the core entities), then the read-only summary and analytics endpoints, then the remaining features.

3 Places cannot be created or edited by anyone

BLOCKER

The place (store) create and edit form is the largest screen in the dashboard — seven sections covering basic info, location and map pin, contact details, working hours, subscription, photo and video gallery, and a product list. It is fully built and validates its input.

It saves nothing. The submit handler in the code is stubbed with the comment "*The write endpoint is not built yet, so saving only validates and confirms.*" There is no create or update method in the place repository at all, so unlike the other gaps this one is missing on **both** sides.

The API does have `POST /admin/stores` and `POST /admin/stores/{id}`, but they cover only the basic fields — there is no route for the photo gallery or the videos at all. Working hours and products do have routes, but only as separate per-record calls, while the form saves everything in one submit. Section 4 has a subsection on this.

What to do: Backend: build the place write endpoints — numbers 13, 14 and 15 in section 3 — including nested working hours, media and products. Frontend: wire the form to them.

4 Unvalidated input returns 500 and leaks the database

HIGH

`POST /admin/stores` and `POST /admin/stores/{id}` pass the `type` field straight to MySQL without validating it. Any value outside the column's enum throws a raw SQL error. The value `type=supermarket` — which is what the current Postman collection itself sends for the store-update request — triggers it, so that saved request fails as written.

Two separate problems: the missing validation, and debug mode being on in production. The error response leaks the full SQL statement, the database name, the host and the port. Every other create endpoint validates correctly and returns `422`, so this looks like one field that was missed.

```
POST /admin/stores/2 (_method=PUT, type=supermarket)
--> 500 Internal Server Error
```

```
SQLSTATE[01000]: Warning: 1265 Data truncated for column 'type' at row 1
(Connection: mysql, Host: mysql, Port: 3306, Database: maptop,
SQL: update `stores` set `type` = supermarket, ...)
```

A deliberately invalid value behaves the same way:

```
POST /admin/stores/2 (type=garbage_value) --> 500, same SQL leak
```

What to do: Add an enum validation rule on ``type`` so a bad value returns `422`, and turn off `APP_DEBUG` in production so SQL and connection details are never returned to a client.

3. Endpoint reference

Paths are relative to the API base URL. The dashboard is configured with one base URL and calls every path below beneath it.

Authentication

1 endpoints

One endpoint. The dashboard signs in, keeps the returned token, and sends it as a bearer token on every other call.

1 **POST** /auth/login

NEEDS CHANGES

Sign an admin in and return the token plus the fields the top bar shows.

Existing endpoint: **POST** /admin/login

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
email	string	yes	Admin email.
password	string	yes	Plain text, over HTTPS.

RESPONSE 200 OK

```
{
  "id": "1",
  "name": "Obedah",
  "role": "super_admin",
  "avatarUrl": null,
  "token": "19|IMIsakH3jSYZAYYrrTvsc6rEpQX4Fad1..."
}
```

Note. Works today but the shape differs. The API nests the user under data.user and the token under data.token; the dashboard expects one flat object with exactly these five fields. It also needs avatarUrl, which the API does not return at all. Return null when the admin has no picture. A wrong password correctly returns 401.

Dashboard (home screen)

5 endpoints

The landing screen: six headline numbers, a list of things needing attention, the newest places, and two charts. All read-only. None of these exist today.

2 GET /dashboard/summary

NOT BUILT

The six numbers across the top of the home screen.

RESPONSE 200 OK

```
{
  "revenue": 4820000,
  "pendingReviewCount": 7,
  "placeCount": 412,
  "newPlaceCount": 18,
  "userCount": 9630,
  "newUserCount": 214
}
```

Note. revenue is a number in the platform currency, not a formatted string. The two 'new' counts are for the current month.

3 GET /dashboard/action-items

NOT BUILT

The 'needs your attention' list — each row links the admin to a filtered screen.

RESPONSE 200 OK

```
[
  { "id": "places-pending", "label": "أماكن بانتظار الموافقة", "count": 12, "tone": "warning" },
  { "id": "reviews-reported", "label": "تقييمات مبلّغ عنها", "count": 3, "tone": "error" }
]
```

Note. tone is one of error | warning | info | success and drives the row colour. Return the rows in the order you want them shown.

4 GET /dashboard/recent-places

NOT BUILT

The newest places table on the home screen.

RESPONSE 200 OK

```
[
  {
    "id": "402",
    "name": "مطعم الشام",
    "category": "مطاعم",
    "city": "طرطوس",
    "rating": 4.6,
    "status": "pending",
    "joinedAt": "2026-09-25T10:14:00Z",
    "logoUrl": "https://api.mapmob.com.co/storage/stores/logos/ab12.png"
  }
]
```

Note. status is active | suspended | pending. Return about 5-10 rows; the screen does not page this table.

5 GET /dashboard/revenue

NOT BUILT

Revenue chart for the home screen.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
period	string	yes	daily weekly monthly yearly

RESPONSE 200 OK

```
{
  "name": "الإيرادات",
  "points": [
    { "label": "يناير", "value": 320000 },
    { "label": "فبراير", "value": 415000 }
  ]
}
```

Note. label is the x-axis text, already formatted for the chosen period — the dashboard prints it as given and does no date maths. One series.

6 GET /dashboard/growth

NOT BUILT

Growth chart for the home screen — several series on one axis.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
period	string	yes	daily weekly monthly yearly

RESPONSE 200 OK

```
[
  { "name": "مستخدمون", "points": [ { "label": "يناير", "value": 820 } ] },
  { "name": "أماكن", "points": [ { "label": "يناير", "value": 46 } ] }
]
```

Note. Same point shape as the revenue chart, but an array of series. Every series should carry the same labels in the same order.

The core entity — what the API calls a store. The list, the detail page, bulk actions, CSV export, and the seven-section create/edit form. The form is the single biggest gap in the project: it is fully built and saves nothing.

7 GET /places

NEEDS CHANGES

The paged places table.

Existing endpoint: GET /admin/stores

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
status	string	no	active suspended pending
category	string	no	Category name as shown in the filter.
package	string	no	free basic premium
sort	string	no	newest oldest rating name — note this list has a 'rating' option the other screens do not.

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "12",
      "code": "PL-0012",
      "name": "متجر دمشق المركزي",
      "logoUrl": "https://api.mapmob.com.co/storage/stores/logos/ab12.png",
      "category": "مواد غذائية",
      "city": "دمشق",
      "rating": 4.6,
      "status": "active",
      "package": "free",
      "reviewCount": 84,
      "joinedAt": "2026-09-22T19:01:50Z"
    }
  ],
  "totalCount": 412
}
```

Note. The list exists but is missing code (a human-readable reference the table shows), rating, package (the subscription tier) and joinedAt, and none of the filters or sorting are supported. It also returns the Laravel {data, links, meta} envelope rather than {items, totalCount}.

8 GET /places/status-counts

NOT BUILT

Counts for the filter tabs above the table, so each tab can show a number.

RESPONSE 200 OK

```
{ "all": 412, "active": 351, "suspended": 24, "pending": 37 }
```

Note. These are counts across the whole table, ignoring the current page but respecting nothing else — they do not change when the admin filters.

9 GET /places/{id}

NEEDS CHANGES

The full place detail page, and the source the edit form loads from.

Existing endpoint: GET /admin/stores/{id}

RESPONSE 200 OK

```
{
  "id": "12", "code": "PL-0012", "name": "متجر دمشق", "status": "active",
  "description": "متجر لبيع المواد الغذائية",
  "mainCategory": "مواد غذائية", "subCategory": "سوبر ماركت",
  "images": ["https://.../1.png", "https://.../2.png"],
  "isOpenNow": true,
  "owner": { "name": "Obedah", "phone": "0931234567", "extraPhone": "" },
  "subscription": { "package": "free", "status": "active", "renewsAt": "2027-01-01" },
  "activity": { "addedAt": "2026-09-22T19:01:50Z", "updatedLabel": "منذ 3 أيام" },
  "contact": {
    "phone": "0931234567", "extraPhone": "", "website": "https://example.com",
    "whatsapp": "0931234567", "facebook": "", "instagram": "", "telegram": ""
  },
  "location": {
    "city": "دمشق", "region": "المركز",
    "address": "شارع الجمهورية",
    "latitude": 34.889, "longitude": 35.8866
  },
  "workingHours": [
    { "days": "السبت - الخميس", "hours": "09:00 - 18:00", "isToday": true }
  ],
  "products": [
    { "id": "5", "name": "مرتديلا", "price": 100, "currency": "SYP",
      "imageUrl": "https://.../p.png", "isAvailable": true, "orderUrl": "" }
  ],
  "offers": [
    { "id": "2", "title": "عرض", "description": "...", "dateRange": "22/09 - 22/12",
      "imageUrl": "https://.../o.png", "isActive": true }
  ],
  "videos": [
    { "id": "1", "url": "https://.../v.mp4", "posterUrl": "https://.../v.jpg", "duration": "0:42" }
  ]
}
```

Note. The existing endpoint returns the basic fields but none of the nested blocks: no products, offers, videos, images gallery, working hours, isOpenNow, subscription or activity. Two fields are pre-formatted strings on purpose — activity.updatedLabel and workingHours[].days/hours are printed as given, because the grouping rule ('Saturday to Thursday') is a business rule, not a display rule. isToday marks the row to highlight.

10 **PATCH** `/places/status`**NOT BUILT**

Activate or suspend several places at once, from the table's bulk action bar.

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
ids	string[]	yes	Place ids to change.
status	string	yes	active suspended

RESPONSE 204 NO CONTENT

(empty body)

Note. Note the path has no id — this is a bulk operation on the collection. The dashboard reloads the table afterwards, so nothing needs to be returned.

11 **POST** `/places/delete`**NOT BUILT**

Delete several places at once from the bulk action bar.

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
ids	string[]	yes	Place ids to delete.

RESPONSE 204 NO CONTENT

(empty body)

Note. POST rather than DELETE because it carries a body of ids. A single-place DELETE `/admin/stores/{id}` already exists and works, but the dashboard only ever deletes from a multi-select, so the bulk form is what it needs.

12 **GET** `/places/export`**NOT BUILT**

Download the filtered table as a CSV file.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
status	string	no	Same filters as the list.
category	string	no	
package	string	no	
ids	string[]	no	Repeated. When present, export only these rows and ignore the filters.

RESPONSE 200 OK

CSV file bytes, Content-Type: text/csv

Note. Return the file as a download, not JSON — the dashboard reads the response as a blob. Ignore `pageIndex/pageSize` and export every matching row. Same pattern for all six `/export` endpoints.

Create a place from the seven-section form. This does not exist on either side yet.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	Place name.
ownerName	string	no	Owner's name.
ownerPhone	string	yes	Owner's phone.
ownerExtraPhone	string	no	
mainCategory	string	yes	Main category. See the note about ids.
subCategory	string	no	Sub category.
city	string	yes	City / area.
region	string	yes	Governorate / region.
address	string	yes	Street address.
latitude	number	yes	Map pin, set from the map picker.
longitude	number	yes	Map pin.
phone	string	yes	Public phone.
extraPhone	string	no	
website	string	no	
whatsapp	string	no	
facebook	string	no	
instagram	string	no	
telegram	string	no	
description	string	no	Free text.
package	string	yes	free basic premium
status	string	yes	active suspended pending — a new place defaults to pending.
workingHours[n][day]	string	yes	One entry per weekday, 7 in total.
workingHours[n][isOpen]	boolean	yes	Closed days still send a row.
workingHours[n][opensAt]	string	yes	HH:mm
workingHours[n][closesAt]	string	yes	HH:mm
images[]	file[]	no	Gallery photos. Limit depends on package: free 3, basic 30, premium 100.
videos[]	file[]	no	Gallery videos. free 1, basic 5, premium 20.
products[n][name]	string	no	Product rows, added inline on the form.
products[n][price]	number	no	
products[n][currency]	string	no	SYP USD
products[n][isAvailable]	boolean	no	
products[n][orderUrl]	string	no	
products[n][image]	file	no	

RESPONSE 201 CREATED

The created place, in the same shape as GET /places/{id}.

Note. The form currently sends mainCategory, city and region as Arabic names picked from lists that are hardcoded in the frontend. That is a frontend shortcut that needs fixing: please expose the form-options endpoint below and accept mainCategoryId, cityId and regionId instead. Enforce the per-package media limits server-side and return 422 when exceeded — the frontend checks them too, but it should not be the only guard.

14 PUT /places/{id}

NOT BUILT

Update a place from the same form.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
(all fields from POST /places)			Same body as create.
keptImageUrls[]	string[]	no	URLs of existing gallery photos to keep. Anything absent is deleted.
keptVideoUrls[]	string[]	no	Same for videos.
products[n][id]	string	no	Present for an existing product, absent for a new one.

RESPONSE 200 OK

The updated place, in the same shape as GET /places/{id}.

Note. The existing POST /admin/stores/{id} covers the basic fields only, and is the endpoint with the 500 bug in issue 4. Editing needs the keep-lists so the admin can remove one photo without re-uploading the rest.

15 GET /places/form-options

NOT BUILT

The dropdown contents for the place form: categories, governorates and their areas, and packages.

RESPONSE 200 OK

```
{
  "mainCategories": [
    { "id": "1", "name": "مواد غذائية",
      "subCategories": [ { "id": "7", "name": "سوبر ماركت" } ] }
  ],
  "governorates": [
    { "id": "1", "name": "دمشق", "areas": [ { "id": "2", "name": "داريا" } ] }
  ],
  "packages": [ { "id": "free", "name": "مجاني" } ]
}
```

Note. Needed to remove the hardcoded Arabic category, city and region lists currently sitting in the place form component. Without this the form can only offer five fake categories.

16 GET /places/{id}/offer-items

NOT BUILT

The place's products, used by the offer form when an offer applies to selected items only.

Existing endpoint: GET /admin/products (partly)

RESPONSE 200 OK

```
[  
  { "id": "5", "name": "مرتديلا هنا", "price": 100, "currency": "SYP" }  
]
```

Note. GET /admin/products returns all products across all stores. This needs the products of one place, and only four fields.

Categories

6 endpoints

Two-level category tree — main categories and their children. Mostly exists, but the icon model is fundamentally different from what the dashboard uses.

17 GET /categories

NEEDS CHANGES

The paged category table, plus the counts for the three filter chips.

Existing endpoint: GET /admin/categories

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
kind	string	no	main sub
status	string	no	active suspended
parentId	string	no	Children of one main category.

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "1",
      "name": "مطاعم",
      "kind": "main",
      "parentId": null,
      "parentName": null,
      "icon": "utensils-crossed",
      "color": "#FF8104",
      "placeCount": 34,
      "status": "active",
      "updatedAt": "2026-09-22T19:01:50Z"
    }
  ],
  "totalCount": 61,
  "kindCounts": { "all": 61, "main": 12, "sub": 49 }
}
```

Note. Note kindCounts sits alongside items and totalCount on this endpoint — it is the only list endpoint with an extra field, because the chips above the table show counts. Missing today: kind (derive it from parentId), placeCount, and the counts. is_active must become status, and the localised name object must be resolved to a string.

18 GET /categories/main

NOT BUILT

Just the main categories, for the parent dropdown when adding a sub category.

RESPONSE 200 OK

```
[ { "id": "1", "name": "مطاعم" } ]
```

Note. Two fields only. GET /categories?kind=main would also work, but the dashboard calls this dedicated path.

19 **POST** /categories

NEEDS CHANGES

Create a category.

Existing endpoint: POST /admin/categories

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	Category name.
kind	string	yes	main sub
parentId	string null	yes	The parent id when kind is sub, null when main.
icon	string	yes	An icon NAME, not a file. See the note.
color	string	yes	Hex colour from the fixed palette, e.g. #FF8104.

RESPONSE 201 CREATED

The created category, in the list item shape above.

Note. This is the important mismatch. The API treats the icon as an uploaded image file and stores a URL. The dashboard uses a fixed set of 60 built-in Lucide icons and sends the icon's NAME as a string (utensils-crossed, coffee, pill, ...), because the icons ship inside the app and are drawn in the chosen colour. Please store icon as a plain string. The colour field already matches the API's icon_color, just renamed. The existing endpoint also takes JSON here rather than multipart, since there is no longer a file.

20 **PUT** /categories/{id}

NEEDS CHANGES

Update a category.

Existing endpoint: POST /admin/categories/{id}

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(same as POST /categories)			

RESPONSE 200 OK

The updated category.

Note. Exists, but needs the real PUT verb instead of _method=PUT, and the same icon change.

21 **PATCH** /categories/{id}/status

NOT BUILT

Toggle one category between active and suspended from the table's row menu.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	active suspended

RESPONSE 200 OK

The updated category.

Note. A small dedicated endpoint so the row toggle does not have to send the whole record back.

22 **DELETE** /categories/{id}

READY

Delete a category.

Existing endpoint: DELETE /admin/categories/{id}

RESPONSE 204 NO CONTENT

(empty body)

Note. Works today. Please decide and document what happens to a main category that still has children or places — reject with 422, or cascade.

Regions

11 endpoints

Governorates and the areas inside them. The API already has this as regions and cities — the same two-level idea under different names. Everything here maps onto existing endpoints except the status toggles.

23 GET /governorates

NEEDS CHANGES

The paged governorate table.

Existing endpoint: GET /admin/regions

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "1",
      "name": "محافظة دمشق",
      "subAreaCount": 8,
      "placeCount": 126,
      "status": "active",
      "updatedAt": "2026-09-22T19:01:23Z"
    }
  ],
  "totalCount": 14
}
```

Note. Maps to /admin/regions. The API already returns cities_count and stores_count, which are exactly subAreaCount and placeCount — just renamed. Needs status instead of is_active, the resolved name string, and the {items, totalCount} envelope.

24 GET /governorates/{id}

NEEDS CHANGES

One governorate, for the edit dialog.

Existing endpoint: GET /admin/regions/{id}

RESPONSE 200 OK

One governorate in the shape above.

Note. Exists. Same field renames.

25 **POST** `/governorates`**NEEDS CHANGES**

Create a governorate.

Existing endpoint: `POST /admin/regions`

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	Governorate name.
status	string	yes	active suspended

RESPONSE 201 CREATED

The created governorate.

Note. Exists. The dashboard sends only name and status — it does not send slug or sortOrder, so please generate the slug server-side and let sort order default.

26 **PUT** `/governorates/{id}`**NEEDS CHANGES**

Update a governorate.

Existing endpoint: `POST /admin/regions/{id}`

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
status	string	yes	active suspended

RESPONSE 200 OK

The updated governorate.

Note. Exists; needs the real PUT verb.

27 **PATCH** `/governorates/{id}/status`**NOT BUILT**

Toggle one governorate active or suspended.

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	active suspended

RESPONSE 200 OK

The updated governorate.

28 DELETE /governorates/{id}**READY**

Delete a governorate.

Existing endpoint: DELETE /admin/regions/{id}

RESPONSE 204 NO CONTENT

(empty body)

Note. Works today. Please document what happens when it still holds areas or places.

29 GET /governorates/{governorateId}/areas**NEEDS CHANGES**

The paged areas inside one governorate.

Existing endpoint: GET /admin/cities

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "2", "governorateId": "1", "name": "داريا",
      "subAreaCount": 0, "placeCount": 31, "status": "active",
      "updatedAt": "2026-09-22T19:01:39Z"
    }
  ],
  "totalCount": 8
}
```

Note. GET /admin/cities exists but returns a flat list of all cities. The dashboard needs them nested under a governorate, as this path. Filtering the existing endpoint by region would also work if the path is kept.

30 POST /governorates/{governorateId}/areas**NEEDS CHANGES**

Create an area inside a governorate.

Existing endpoint: POST /admin/cities

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
status	string	yes	active suspended

RESPONSE 201 CREATED

The created area.

Note. Exists as /admin/cities with region_id in the body; here the governorate comes from the path.

31 **PUT** `/areas/{id}`**NEEDS CHANGES**

Update an area.

Existing endpoint: `POST /admin/cities/{id}`

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
status	string	yes	active suspended

RESPONSE 200 OK

The updated area.

Note. Exists; needs the real PUT verb. Note the path is top-level `/areas/{id}`, not nested.

32 **PATCH** `/areas/{id}/status`**NOT BUILT**

Toggle one area active or suspended.

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	active suspended

RESPONSE 200 OK

The updated area.

33 **DELETE** `/areas/{id}`**READY**

Delete an area.

Existing endpoint: `DELETE /admin/cities/{id}`

RESPONSE 204 NO CONTENT

(empty body)

Note. Works today.

App users — registered accounts and anonymous visitors. The list and single-user reads exist; the summary, the status toggle, delete and export do not.

34 GET /users

NEEDS CHANGES

The paged users table.

Existing endpoint: GET /admin/users

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
accountType	string	no	registered visitor
status	string	no	active suspended
registeredFrom	string	no	YYYY-MM-DD, inclusive.
registeredTo	string	no	YYYY-MM-DD, inclusive.

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "5",
      "name": "obedah1",
      "email": null,
      "phone": "963936318327",
      "accountType": "registered",
      "governorateName": "دمشق",
      "registeredAt": "2026-09-25T18:01:25Z",
      "lastActiveAt": "2026-09-27T03:05:43Z",
      "status": "active"
    }
  ],
  "totalCount": 9630
}
```

Note. GET /admin/users exists and works but is not in the Postman collection — please add it. Missing fields: accountType, governorateName, lastActiveAt, and status (the API returns roles and email_verified_at instead). email and phone are both nullable; a visitor account may have neither. None of the filters are supported yet.

35 GET /users/summary

NOT BUILT

The four cards above the users table.

RESPONSE 200 OK

```
{
  "totalUserCount": 9630,
  "activeUserCount": 9310,
  "suspendedUserCount": 106,
  "newUserCount": 214
}
```

Note. newUserCount is this month. Counts are across the whole table, not the current filter.

36 GET /users/{id}

NEEDS CHANGES

The user detail page: the user, their activity stats, a recent activity feed, their saved places and their reviews.

Existing endpoint: GET /admin/users/{id}

RESPONSE 200 OK

```
{
  "user": { ... same shape as a row in GET /users ... },
  "stats": {
    "searchCount": 184, "viewedPlaceCount": 92,
    "favoritePlaceCount": 14, "reviewCount": 6
  },
  "activities": [
    { "id": "a1", "type": "search", "description": "بحث عن مطاعم",
      "occurredAt": "2026-09-26T12:02:00Z" }
  ],
  "favoritePlaces": [
    { "id": "f1", "placeName": "مطعم الشام", "placeKind": "restaurant",
      "categoryName": "مطاعم", "governorateName": "دمشق",
      "savedAt": "2026-09-20T10:00:00Z" }
  ],
  "reviews": [
    { "id": "r1", "placeName": "مطعم الشام", "categoryName": "مطاعم",
      "locationName": "دمشق", "rating": 5, "comment": "ممتاز",
      "reviewedAt": "2026-09-21T09:30:00Z" }
  ]
}
```

Note. A single user read exists but returns only the user record — none of the four nested blocks. activities[].type is search | favorite | review | share. placeKind is restaurant | pharmacy | other.

37 PATCH /users/{id}/status

NOT BUILT

Suspend or reactivate one user.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	active suspended

RESPONSE 200 OK

The updated user row.

38 **DELETE** /users/{id}

NOT BUILT

Delete a user account.

RESPONSE 204 NO CONTENT

(empty body)

Note. Please confirm whether this should be a soft delete, given the user's reviews and favourites reference them.

39 **GET** /users/export

NOT BUILT

Download the filtered users table as CSV.

QUERY PARAMETERS

same as GET /users

RESPONSE 200 OK

CSV file bytes.

Note. Ignore paging and export every matching row.

Reviews

8 endpoints

Star ratings and comments left on places, plus the moderation queue for reviews that users have reported. Nothing in this feature exists yet — all eight endpoints are new.

40 GET /reviews

NOT BUILT

The paged reviews table.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
rating	string	no	fiveStars fourStarsAndUp threeStars twoStars oneStar unrated
status	string	no	published reported hidden
placeName	string	no	Filter by place.
submittedFrom	string	no	YYYY-MM-DD
submittedTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "r1",
      "userName": "obedah1",
      "placeName": "مطعم الشام",
      "rating": 5,
      "comment": "خدمة ممتازة",
      "createdAt": "2026-09-21T09:30:00Z",
      "status": "published"
    }
  ],
  "totalCount": 1284
}
```

Note. rating is nullable — a user may leave a comment with no stars, which is what the 'unrated' filter selects. The rating filter values are named buckets, not numbers: fourStarsAndUp means 4 or 5.

41 GET /reviews/summary

NOT BUILT

The four cards above the reviews table.

RESPONSE 200 OK

```
{
  "totalCount": 1284,
  "averageRating": 4.3,
  "newCount": 37,
  "reportedCount": 6
}
```

Note. averageRating is across all rated reviews, excluding unrated ones. newCount is this month.

42 GET /reviews/{id}

NOT BUILT

One review with its place, its author, and the report against it when there is one.

RESPONSE 200 OK

```
{
  "review": { ... same shape as a row in GET /reviews ... },
  "place": {
    "referenceNumber": 12, "name": "مطعم الشام",
    "categoryName": "مطاعم",
    "governorateName": "دمشق", "areaName": "داريا"
  },
  "reviewer": {
    "name": "obedah1", "memberSince": "2026-09-25T18:01:25Z",
    "reviewCount": 6, "isPhoneVerified": true
  },
  "report": {
    "reporterName": "محمد",
    "reason": "تقييم مزيف",
    "notes": "..."
  }
}
```

Note. report is null when nobody reported the review. referenceNumber is a number here, unlike ids elsewhere — it is the place's human-facing reference shown on the page.

43 POST /reviews/{id}/report/accept

NOT BUILT

Accept a report: the reporter was right, so hide the review.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			The dashboard sends an empty body.

RESPONSE 200 OK

The updated review row, with status now hidden.

Note. Accepting a report should set the review's status to hidden and clear it from the reported queue.

44 POST /reviews/{id}/report/reject

NOT BUILT

Reject a report: the review is fine, so publish it again.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 200 OK

The updated review row, with status back to published.

Note. Should set status to published and clear the report.

45 **PATCH** `/reviews/{id}/status`**NOT BUILT**

Hide or publish a review directly, without going through a report.

REQUEST BODY `application/json`

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	published hidden — 'reported' is not settable by an admin.

RESPONSE 200 OK

The updated review row.

Note. Only these two values are accepted here; 'reported' is a state users create, not admins.

46 **DELETE** `/reviews/{id}`**NOT BUILT**

Delete a review permanently.

RESPONSE 204 NO CONTENT

(empty body)

Note. Deleting should also update the place's rating average and review count.

47 **GET** `/reviews/export`**NOT BUILT**

Download the filtered reviews table as CSV.

QUERY PARAMETERS

same as GET /reviews

RESPONSE 200 OK

CSV file bytes.

Offers

10 endpoints

Discount campaigns attached to a place, optionally limited to certain products. Create, update, delete and the list exist; the summary, the pause/resume actions, the form dropdowns and the export do not.

48 GET /offers

NEEDS CHANGES

The paged offers table.

Existing endpoint: GET /admin/offers

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
status	string	no	active scheduled paused expired draft
runningFrom	string	no	YYYY-MM-DD — offers running on or after this date.
runningTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "2",
      "title": "عرض الشهر",
      "placeName": "متجر دمشق",
      "categoryName": "مواد غذائية",
      "startsOn": "2026-09-22",
      "endsOn": "2026-12-22",
      "status": "active"
    }
  ],
  "totalCount": 48
}
```

Note. Exists. The status vocabulary is the important difference: the API stores active/inactive, but the dashboard shows five states. scheduled (starts in the future), active (running now) and expired (past its end date) can all be derived from the dates plus a paused flag and a draft flag — please compute status server-side rather than making the dashboard do date maths. placeName and categoryName are also needed as flat strings.

49 GET /offers/summary

NOT BUILT

The four cards above the offers table.

RESPONSE 200 OK

```
{ "totalCount": 48, "activeCount": 22, "scheduledCount": 9, "endedCount": 17 }
```

Note. The same four-field shape is used by the ads summary — they share one model on the frontend, so please keep the field names identical for both.

50 GET /offers/{id}

NEEDS CHANGES

One offer with everything the edit form needs.

Existing endpoint: GET /admin/offers/{id}

RESPONSE 200 OK

```
{
  "offer": { ... same shape as a row in GET /offers ... },
  "description": "عرض الشهر المميز",
  "place": {
    "id": "2", "name": "متجر دمشق",
    "categoryName": "مواد غذائية",
    "address": "شارع الجمهورية"
  },
  "discountPercent": 20,
  "scope": "selectedItems",
  "itemIds": ["5", "9"],
  "imageUrl": "https://api.mapmob.com.co/storage/offers/abc.png"
}
```

Note. Exists but flat. The dashboard needs the offer nested under an 'offer' key plus these extra fields. scope is allItems | selectedItems; itemIds is empty when scope is allItems.

51 POST /offers/{id}/pause

NOT BUILT

Pause a running offer.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 200 OK

The updated offer row, status now paused.

52 POST /offers/{id}/resume

NOT BUILT

Resume a paused offer.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 200 OK

The updated offer row, status back to active or scheduled depending on its dates.

Note. Resuming should recompute the status from the dates, not blindly set active — a resumed offer whose start date is still in the future is scheduled.

53 GET /offers/form-options

NOT BUILT

Dropdown contents for the offer form.

RESPONSE 200 OK

```
{
  "places": [
    { "id": "2", "name": "متجر دمشق",
      "categoryName": "مواد غذائية", "address": "شارع الجمهورية" },
  ],
  "categoryNames": ["مطاعم", "مواد غذائية"]
}
```

Note. The form shows the place's category and address once a place is picked, which is why places carry more than id and name here.

54 POST /offers

NEEDS CHANGES

Create an offer.

Existing endpoint: POST /admin/offers

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
title	string	yes	
discountPercent	number	yes	Percent, 0-100.
placeId	string	yes	The place the offer belongs to.
categoryName	string	yes	
startsOn	string	yes	YYYY-MM-DD
endsOn	string	yes	YYYY-MM-DD
status	string	yes	active paused draft — only these three are settable.
description	string	yes	
scope	string	yes	allItems selectedItems
itemIds	string[]	no	Repeated field. Sent only when scope is selectedItems.
image	file	no	Offer image.
isImageRemoved	boolean	yes	true to clear the existing image.

RESPONSE 201 CREATED

The created offer row.

Note. Exists but the fields differ. The API takes original_price / discounted_price / discount_percentage / currency_code / is_featured and a product_id; the dashboard sends a discount percent, a scope and a list of item ids instead, and has no notion of featured offers or per-offer currency. Please align on these fields. Note status here excludes scheduled and expired, which are computed, not chosen.

55 **PUT** `/offers/{id}`**NEEDS CHANGES**

Update an offer.

Existing endpoint: `POST /admin/offers/{id}`

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
(same as <code>POST /offers</code>)			

RESPONSE 200 OK

The updated offer row.

Note. Exists; needs the real PUT verb and the field alignment above.

56 **DELETE** `/offers/{id}`**READY**

Delete an offer.

Existing endpoint: `DELETE /admin/offers/{id}`

RESPONSE 204 NO CONTENT

(empty body)

Note. Works today, though it is not in the Postman collection — please add it.

57 **GET** `/offers/export`**NOT BUILT**

Download the filtered offers table as CSV.

QUERY PARAMETERS

same as `GET /offers`

RESPONSE 200 OK

CSV file bytes.

Paid banner campaigns placed around the app. Structurally a near-twin of offers. The list, create, update and delete exist; the summary, pause/resume, form dropdowns, per-ad metrics and export do not.

58 GET /ads

NEEDS CHANGES

The paged ads table.

Existing endpoint: GET /admin/ads

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
status	string	no	active scheduled paused expired draft
contentType	string	no	image video
advertiserType	string	no	place admin
placement	string	no	home searchResults categories placeDetails
runningFrom	string	no	YYYY-MM-DD
runningTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "2",
      "title": "اعلان الموسم",
      "advertiserType": "place",
      "placeName": "متجر دمشق",
      "contentType": "image",
      "placement": "home",
      "priority": 5,
      "startsOn": "2026-09-22",
      "endsOn": "2026-12-22",
      "status": "active"
    }
  ],
  "totalCount": 31
}
```

Note. Exists. Differences: advertiserType (place means a company paid for it, admin means the platform ran it), contentType, and priority as an integer 1-5 where 5 shows first. placeName is null when advertiserType is admin. placement values are camelCase — the API currently stores 'home' only.

59 GET /ads/summary

NOT BUILT

The four cards above the ads table.

RESPONSE 200 OK

```
{ "totalCount": 31, "activeCount": 14, "scheduledCount": 5, "endedCount": 12 }
```

Note. Identical shape to the offers summary — same four field names.

60 GET /ads/{id}

NEEDS CHANGES

One ad with its body text, media, audit fields and performance metrics.

Existing endpoint: GET /admin/ads/{id}

RESPONSE 200 OK

```
{
  "ad": { ... same shape as a row in GET /ads ... },
  "placeId": "2",
  "position": "topBanner",
  "text": "اعلان اعلان",
  "mediaUrl": "https://api.mapmob.com.co/storage/ads/images/abc.png",
  "createdOn": "2026-09-27T17:59:56Z",
  "updatedOn": "2026-09-27T18:00:30Z",
  "updatedBy": "0bedah",
  "metrics": { "impressions": 18420, "clicks": 642, "uniqueUsers": 9130 }
}
```

Note. Exists but is missing position (topBanner | middleBanner | bottomBanner), the audit trio createdOn/updatedOn/updatedBy, and metrics. The API already counts views_count and clicks_count, which are impressions and clicks renamed — uniqueUsers is new. placeId is null for an admin ad.

61 POST /ads/{id}/pause

NOT BUILT

Pause a running ad.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 200 OK

The updated ad row, status now paused.

62 POST /ads/{id}/resume

NOT BUILT

Resume a paused ad.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 200 OK

The updated ad row, status recomputed from its dates.

63 GET /ads/form-options

NOT BUILT

The place dropdown for the ad form.

RESPONSE 200 OK

```
{ "places": [ { "id": "2", "name": "متجر دمشق" } ] }
```

Note. Only needed when advertiserType is place. Two fields, unlike the offers version.

64 POST /ads

NEEDS CHANGES

Create an ad.

Existing endpoint: POST /admin/ads

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
title	string	yes	
advertiserType	string	yes	place admin
placeId	string	no	Required when advertiserType is place; omitted for admin.
contentType	string	yes	image video
text	string	yes	Banner body text.
placement	string	yes	home searchResults categories placeDetails
position	string	yes	topBanner middleBanner bottomBanner
startsOn	string	yes	YYYY-MM-DD
endsOn	string	no	YYYY-MM-DD, or omitted for an open-ended ad.
priority	integer	yes	1-5, where 5 shows first.
status	string	yes	active paused draft
media	file	no	One image or video, matching contentType.
isMediaRemoved	boolean	yes	true to clear the existing media.

RESPONSE 201 CREATED

The created ad row.

Note. Exists but takes an indexed media array (media[0][type], media[0][file], media[0][sort_order]) allowing several files. The dashboard sends exactly one file in a single 'media' field plus a contentType, and has no sort order. It also needs position and priority, which the API does not store, and target_url is not a field the dashboard collects. Please align on this list.

65 **PUT** /ads/{id}

NEEDS CHANGES

Update an ad.

Existing endpoint: POST /admin/ads/{id}

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
(same as POST /ads)			

RESPONSE 200 OK

The updated ad row.

Note. Exists; needs the real PUT verb and the field alignment above.

66 **DELETE** /ads/{id}

READY

Delete an ad.

Existing endpoint: DELETE /admin/ads/{id}

RESPONSE 204 NO CONTENT

(empty body)

Note. Works today.

67 **GET** /ads/export

NOT BUILT

Download the filtered ads table as CSV.

QUERY PARAMETERS

same as GET /ads

RESPONSE 200 OK

CSV file bytes.

Subscriptions and plans

7 endpoints

The package tiers the platform sells, and the record of which company is on which plan. Two screens behind one tabbed page. Nothing here exists yet.

68 GET /subscriptions/summary

NOT BUILT

The cards above the subscriptions page.

RESPONSE 200 OK

```
{
  "topPlanName": "الباقة المميزة",
  "topPlanSubscriberCount": 148,
  "availablePlanCount": 3,
  "endingSoonCount": 12,
  "activeSubscriberCount": 268,
  "activeSubscriberShare": 0.64
}
```

Note. activeSubscriberShare is a fraction between 0 and 1, not a percentage — the dashboard formats it. endingSoonCount is subscriptions ending within 30 days.

69 GET /subscription-plans

NOT BUILT

The plan cards on the packages tab.

RESPONSE 200 OK

```
[
  {
    "id": "2",
    "name": "الباقة المميزة",
    "tier": "featured",
    "tagline": "للمتاجر الكبيرة",
    "badge": "الأكثر طلباً",
    "monthlyPrice": 250000,
    "yearlyPrice": 2500000,
    "currency": "SYP",
    "subscriberCount": 148,
    "limits": {
      "adsPerMonth": 10, "activeOffers": 20,
      "galleryImages": 100, "videos": 20
    },
    "features": ["أولوية في البحث", "تقارير شهرية"],
    "isActive": true
  }
]
```

Note. tier is featured | basic | free. badge is nullable — only one plan usually carries one. Any value inside limits may be null, which the dashboard renders as unlimited. yearlyPrice is nullable for plans billed monthly only. features is a plain list of strings shown as bullets.

70 **PUT** `/subscription-plans/{id}`**NOT BUILT**

Edit a plan's pricing, limits and feature list.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
isActive	boolean	yes	
monthlyPrice	number	yes	
yearlyPrice	number null	yes	null when the plan has no yearly option.
currency	string	yes	SYP USD
limits	object	yes	The four keys above; null means unlimited.
features	string[]	yes	Replaces the whole list.

RESPONSE 200 OK

The updated plan.

Note. tier is not editable — it is the plan's identity. features replaces the list wholesale rather than patching individual entries.

71 **PATCH** `/subscription-plans/{id}`**NOT BUILT**

Turn a plan on or off without editing it.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
isActive	boolean	yes	

RESPONSE 200 OK

The updated plan.

Note. Note this PATCH is on the plan itself, not a /status sub-path, and its body key is isActive rather than status — plans are on/off, not active/suspended.

72 **DELETE** `/subscription-plans/{id}`**NOT BUILT**

Delete a plan.

RESPONSE 204 NO CONTENT

(empty body)

Note. Should be refused with 422 when companies are still subscribed to it.

73 GET /subscriptions

NOT BUILT

The paged subscription records table.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
tier	string	no	featured basic free
status	string	no	active paused expired
subscribedFrom	string	no	YYYY-MM-DD
subscribedTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "s1",
      "companyName": "متجر دمشق",
      "planName": "الباقة المميزة",
      "planTier": "featured",
      "price": 250000,
      "currencySymbol": "ل.س",
      "startedOn": "2026-01-01",
      "endsOn": "2027-01-01",
      "status": "active"
    }
  ],
  "totalCount": 268
}
```

Note. Note currencySymbol is the symbol itself, not a currency code — this row shows it directly.

74 GET /subscriptions/export

NOT BUILT

Download the filtered subscriptions table as CSV.

QUERY PARAMETERS

same as GET /subscriptions

RESPONSE 200 OK

CSV file bytes.

The record of money received, and a form to enter a payment by hand. Nothing here exists yet.

75 GET /payments**NOT BUILT**

The paged payments table.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
companyName	string	no	Filter by company.
paymentMethod	string	no	cash other
currency	string	no	SYP USD
paidFrom	string	no	YYYY-MM-DD
paidTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "p1",
      "transactionNumber": "TRX-100234",
      "receiptNumber": "RCP-5521",
      "companyName": "متجر دمشق",
      "amount": 250000,
      "currency": "SYP",
      "paymentMethod": "cash",
      "paidAt": "2026-09-20T11:00:00Z",
      "notes": "دفعة سنوية"
    }
  ],
  "totalCount": 512
}
```

Note. The two filter names paidFrom and paidTo are flat query parameters even though the dashboard holds them as a date range object internally.

76 GET /payments/summary**NOT BUILT**

The four cards above the payments table.

RESPONSE 200 OK

```
{
  "pendingCount": 4,
  "transactionCount": 512,
  "monthTotal": 8420000,
  "cumulativeTotal": 96300000
}
```

Note. monthTotal is the current month; cumulativeTotal is all time. Both are numbers in the platform currency.

77 GET /payments/{id}**NOT BUILT**

One payment with the paying company and the subscription it covered.

RESPONSE 200 OK

```
{
  ... all fields from a row in GET /payments ...,
  "company": {
    "name": "متجر دمشق", "type": "مواد غذائية",
    "contactName": "Obedah", "contactPhone": "0931234567"
  },
  "subscription": {
    "planName": "الباقة المميزة", "cycle": "سنوي",
    "startsOn": "2026-01-01", "endsOn": "2027-01-01"
  }
}
```

Note. This one extends the row rather than nesting it — the payment fields sit at the top level alongside company and subscription. subscription is null for a payment not tied to one.

78 GET /payments/form-options**NOT BUILT**

Dropdown contents for the manual payment form.

RESPONSE 200 OK

```
{
  "merchants": [
    {
      "id": "2", "name": "متجر دمشق",
      "currentPlanId": "2", "currentPlanName": "الباقة المميزة",
      "subscriptionEndsOn": "2027-01-01", "currency": "SYP"
    }
  ],
  "plans": [
    { "id": "2", "name": "الباقة المميزة",
      "monthlyPrice": 250000, "yearlyPrice": 2500000 }
  ]
}
```

Note. The merchant's current plan and end date are included so the form can pre-fill the amount and work out whether the payment is a renewal or an upgrade. The three nullable fields are null for a merchant with no subscription yet.

79 **POST** /payments

NOT BUILT

Record a payment by hand.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
merchantId	string	yes	The paying company.
kind	string	yes	new upgrade renewal
planId	string	yes	The plan being paid for.
term	string	yes	monthly yearly
amount	number	yes	
currency	string	yes	SYP USD
paidAt	string	yes	ISO date-time the money arrived.
startsOn	string	yes	YYYY-MM-DD — subscription period start.
endsOn	string	yes	YYYY-MM-DD — subscription period end.
notes	string	no	Free text.

RESPONSE 201 CREATED

The created payment row.

Note. Recording a payment should also create or extend the company's subscription, since the form collects the plan, the term and the period. Please confirm that is the intended behaviour. transactionNumber and receiptNumber are generated server-side — the form does not collect them.

80 **GET** /payments/export

NOT BUILT

Download the filtered payments table as CSV.

QUERY PARAMETERS

same as GET /payments

RESPONSE 200 OK

CSV file bytes.

Analytics reports

5 endpoints

The charts-and-shares reporting screen. Read-only. Careful: the API already uses the word 'reports' for user complaints, which is a different feature — see the naming note in section 4. Nothing here exists yet.

81 GET /reports/category-shares

NOT BUILT

Share of places per category, for the bar list.

RESPONSE 200 OK

```
[
  { "categoryName": "مطاعم", "share": 0.32 },
  { "categoryName": "صيدليات", "share": 0.18 }
]
```

Note. share is a fraction between 0 and 1. Please return them sorted largest first; the dashboard prints them in the given order.

82 GET /reports/governorate-activity

NOT BUILT

Visits per governorate.

RESPONSE 200 OK

```
[
  { "governorateName": "دمشق", "visitCount": 48210, "share": 0.41 }
]
```

Note. Both a raw count and a fraction, because the row shows the number and draws a bar.

83 GET /reports/user-growth

NOT BUILT

User growth chart — several series.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
period	string	yes	daily weekly monthly yearly

RESPONSE 200 OK

```
[
  { "name": "مستخدمون جدد", "points": [ { "label": "يناير", "value": 214 } ] }
]
```

Note. Same series shape as the dashboard charts.

84 GET /reports/usage

NOT BUILT

Feature usage breakdown.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
period	string	yes	daily weekly monthly yearly

RESPONSE 200 OK

```
[
  { "label": "بحوث", "count": 18420, "share": 0.52 },
  { "label": "مفضلة", "count": 6210, "share": 0.18 }
]
```

Note. label is the metric's display name, chosen server-side.

85 GET /reports/revenue

NOT BUILT

Revenue chart on the reports screen — one series.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
period	string	yes	daily weekly monthly yearly

RESPONSE 200 OK

```
{ "name": "الإيرادات", "points": [ { "label": "يناير", "value": 320000 } ] }
```

Note. Same shape as /dashboard/revenue. It can be the same handler if the numbers are the same.

Push notifications

12 endpoints

Composing and sending push notifications to app users or to companies, with targeting by governorate and area or by picking recipients by hand, plus scheduling. Twelve endpoints, none of which exist.

86 GET /notifications

NOT BUILT

The paged notifications table.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
audience	string	no	users companies
kind	string	no	general private
status	string	no	sent scheduled draft
sentFrom	string	no	YYYY-MM-DD
sentTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "n1",
      "title": "تحديث جديد",
      "body": "نسخة جديدة متوفرة",
      "audience": "users",
      "recipientMode": "all",
      "kind": "general",
      "status": "sent",
      "sendAt": "2026-09-26T09:00:00Z",
      "recipientCount": 9310
    }
  ],
  "totalCount": 74
}
```

Note. recipientMode is all | selected | location. kind is derived: 'general' when it went to everyone, 'private' when it went to a chosen list. sendAt is null for a draft.

87 GET /notifications/summary

NOT BUILT

The four cards above the table.

RESPONSE 200 OK

```
{ "totalCount": 74, "sentCount": 58, "scheduledCount": 6, "draftCount": 10 }
```

88 GET /notifications/{id}

NOT BUILT

One notification with everything the edit form needs.

RESPONSE 200 OK

```
{
  ... all fields from a row in GET /notifications ...,
  "imageUrl": "https://api.mapmob.com.co/storage/notifications/abc.png",
  "governorateId": "1",
  "areaId": "2",
  "recipientIds": ["5", "6"]
}
```

Note. Extends the row at the top level. governorateId and areaId are null unless recipientMode is location; recipientIds is empty unless it is selected.

89 POST /notifications

NOT BUILT

Create a notification — send now, schedule it, or save it as a draft.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
title	string	yes	
body	string	yes	
audience	string	yes	users companies
recipientMode	string	yes	all selected location
governorateId	string	no	Sent only when recipientMode is location.
areaId	string	no	Optional even then — a governorate-wide send omits it.
recipientIds	string[]	no	Repeated field. Sent only when recipientMode is selected.
sendAt	string	no	ISO date-time to schedule. Omitted means send immediately.
intent	string	yes	publish draft
image	file	no	Optional image.
isImageRemoved	boolean	yes	

RESPONSE 201 CREATED

The created notification row.

Note. intent is the important field: 'draft' saves without sending regardless of sendAt; 'publish' sends now when sendAt is absent, or schedules when it is present. The resulting status should be draft, scheduled or sent accordingly.

90 PUT /notifications/{id}**NOT BUILT**

Update a notification that has not been sent yet.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
(same as POST /notifications)			

RESPONSE 200 OK

The updated notification row.

Note. Should be refused with 422 once status is sent.

91 DELETE /notifications/{id}**NOT BUILT**

Delete a notification.

RESPONSE 204 NO CONTENT

(empty body)

92 POST /notifications/{id}/duplicate**NOT BUILT**

Copy a notification into a new draft, so the admin can resend it with edits.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(no body)			Empty body.

RESPONSE 201 CREATED

The new notification row, status draft.

Note. The copy should carry over the content and targeting but start as a draft with no sendAt.

93 POST /notifications/{id}/reschedule**NOT BUILT**

Move a scheduled notification to a different time.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
sendAt	string	yes	ISO date-time, must be in the future.

RESPONSE 200 OK

The updated notification row.

Note. Only valid while status is scheduled.

94 **POST** /notifications/{id}/resend

NOT BUILT

Send an already-sent notification again, now or at a chosen time.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
sendAt	string null	yes	null sends immediately; an ISO date-time schedules it.

RESPONSE 200 OK

The notification row that will be sent.

Note. Unlike reschedule, sendAt here is explicitly nullable — the dashboard always sends the key, with null meaning now.

95 **GET** /notifications/form-options

NOT BUILT

Governorates and their areas, for the location targeting picker.

RESPONSE 200 OK

```
{
  "governorates": [
    { "id": "1", "name": "دمشق",
      "areas": [ { "id": "2", "name": "داريا" } ] }
  ]
}
```

Note. Areas are nested so the second dropdown can filter without another call.

96 **GET** /notifications/recipients

NOT BUILT

Search users or companies to pick individual recipients.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
audience	string	yes	users companies
search	string	yes	The typed search text. May be empty for an initial list.

RESPONSE 200 OK

```
[
  { "id": "5", "name": "obedah1", "phone": "963936318327", "city": "دمشق" }
]
```

Note. This is a type-ahead, so it should be fast and capped — around 20 results is plenty.

How many devices a location-targeted send would reach, shown live as the admin picks.

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
audience	string	yes	users companies
governorateId	string	yes	
areaId	string	no	Omitted for a whole-governorate estimate.

RESPONSE 200 OK

```
{ "deviceCount": 4820, "sharePercent": 12.4 }
```

Note. Note sharePercent is a percentage here (12.4 means 12.4%), unlike the 'share' fields elsewhere which are fractions. That is how the model is written, so please match it.

The admin's own notification bell in the top bar. Two endpoints, neither built.

98 GET `/inbox`

NOT BUILT

The admin's notifications, newest first.

RESPONSE 200 OK

```
[
  {
    "id": "i1",
    "category": "complaints",
    "title": "شكوى جديدة",
    "body": "وردت شكوى على متجر دمشق",
    "receivedAt": "2026-09-27T08:12:00Z",
    "isRead": false
  }
]
```

Note. category is complaints | subscriptions | offers | system and drives the icon and the filter tabs. Not paged — return a recent window, say the last 50.

99 PATCH `/inbox/{id}/read`

NOT BUILT

Mark one inbox item as read.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(empty object)			The dashboard sends {}.

RESPONSE 200 OK

The updated inbox item, isRead now true.

Complaints

6 endpoints

Reports users file against a place. This is what the existing API calls 'reports' — the list, the single read and the review update all exist under that name. The summary, delete and export do not.

100 GET /complaints

NEEDS CHANGES

The paged complaints table.

Existing endpoint: GET /admin/reports

QUERY PARAMETERS

NAME	TYPE	REQUIRED	NOTES
pageIndex	integer	yes	Zero-based page number.
pageSize	integer	yes	Rows per page.
search	string	no	Free-text search.
sort	string	no	newest oldest name
status	string	no	new inReview resolved rejected
reportedFrom	string	no	YYYY-MM-DD
reportedTo	string	no	YYYY-MM-DD

RESPONSE 200 OK

```
{
  "items": [
    {
      "id": "1",
      "reference": "CMP-0001",
      "reason": "wrongInformation",
      "status": "new",
      "reportedOn": "2026-09-26T14:20:00Z",
      "reporter": {
        "id": "5", "name": "obedah1",
        "email": "user@example.com", "phone": "963936318327"
      },
      "place": {
        "id": "12", "name": "متجر دمشق",
        "categoryName": "مواد غذائية",
        "address": "شارع الجمهورية",
        "rating": 4.6, "reviewCount": 84,
        "imageUrl": "https://api.mapmob.com.co/storage/stores/logos/ab12.png"
      }
    }
  ],
  "totalCount": 63
}
```

Note. GET /admin/reports exists and already nests the user and the store, which is the right shape. Differences: reason must be one of five fixed codes (wrongInformation | closedPlace | inappropriateContent | fakeReview | other) rather than free Arabic text, status must use the four values above rather than the API's current set, and a human-readable reference is needed. The place block also needs rating, reviewCount and imageUrl.

101 GET /complaints/summary

NOT BUILT

The five cards above the complaints table.

RESPONSE 200 OK

```
{
  "totalCount": 63,
  "newCount": 11,
  "inReviewCount": 8,
  "resolvedCount": 38,
  "rejectedCount": 6
}
```

Note. Five counts here, unlike the four-card summaries elsewhere.

102 GET /complaints/{id}

NEEDS CHANGES

One complaint with its description, attachments and the admin's notes.

Existing endpoint: GET /admin/reports/{id}

RESPONSE 200 OK

```
{
  ... all fields from a row in GET /complaints ...,
  "description": "المعلومات غير صحيحة",
  "userDetails": "2026 عميل منذ",
  "attachmentUrls": ["https://api.mapmob.com.co/storage/complaints/a1.png"],
  "adminNotes": "تم التواصل مع المتجر"
}
```

Note. Exists. Missing attachmentUrls (users can attach photos to a complaint) and userDetails. adminNotes already exists under that name.

103 PATCH /complaints/{id}

NEEDS CHANGES

Record the admin's decision on a complaint.

Existing endpoint: POST /admin/reports/{id}

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
status	string	yes	new inReview resolved rejected
adminNotes	string	yes	May be an empty string.

RESPONSE 200 OK

The updated complaint, in the detail shape above.

Note. Exists and works — it was tested successfully. Needs the real PATCH verb instead of _method=PUT, and the four status values. Note this is a PATCH on the resource itself, not a /status sub-path, because it carries two fields.

104 **DELETE** /complaints/{id}

NOT BUILT

Delete a complaint.

RESPONSE 204 NO CONTENT

(empty body)

105 **GET** /complaints/export

NOT BUILT

Download the filtered complaints table as CSV.

QUERY PARAMETERS

same as GET /complaints

RESPONSE 200 OK

CSV file bytes.

Content pages

11 endpoints

The static pages the mobile app shows: about, terms, privacy, contact, and the FAQ list. Eleven endpoints, none built.

106 GET /content/pages

NOT BUILT

The index of content pages with their status, for the content screen's list.

RESPONSE 200 OK

```
[
  { "kind": "about", "title": "من نحن", "updatedAt": "2026-09-20T10:00:00Z", "status": "published" },
  { "kind": "terms", "title": "الشروط", "updatedAt": "2026-09-18T10:00:00Z", "status": "published" },
  { "kind": "privacy", "title": "الخصوصية", "updatedAt": "2026-09-18T10:00:00Z", "status": "draft" },
  { "kind": "contact", "title": "اتصل بنا", "updatedAt": "2026-09-15T10:00:00Z", "status": "published" },
  { "kind": "faq", "title": "الأسئلة الشائعة", "updatedAt": "2026-09-22T10:00:00Z", "status": "published" }
]
```

Note. kind is the stable key the dashboard routes on: about | terms | privacy | contact | faq. status is published | draft. Return all five rows even when a page has never been edited.

107 GET /content/pages/about

NOT BUILT

The about page content.

RESPONSE 200 OK

```
{
  "title": "من نحن",
  "bannerUrl": "https://api.mapmob.com.co/storage/content/about-banner.png",
  "summary": "<p>منصة ماب موب</p>",
  "phone": "0931234567",
  "email": "info@mapmob.com",
  "address": "دمشق، سوريا"
}
```

Note. summary is HTML — the dashboard edits it in a rich-text editor and sends HTML back. Please store it as-is and sanitise on output for the mobile app.

108 **PUT** **/content/pages/about****NOT BUILT**

Save the about page.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
title	string	yes	
summary	string	yes	HTML from the rich-text editor.
phone	string	yes	
email	string	yes	
address	string	yes	
status	string	yes	published draft
banner	file	no	New banner image.
isBannerRemoved	boolean	yes	true clears the existing banner.

RESPONSE 200 OK

The saved about page, in the GET shape.

Note. This is the only content page that carries a file, which is why it is multipart while the others are JSON.

109 **GET** **/content/pages/{kind}****NOT BUILT**

A legal page's content. {kind} is terms or privacy.

RESPONSE 200 OK

```
{ "title": "الشروط والأحكام", "body": "<p>...</p>" }
```

Note. Two fields only. body is HTML. The same path serves both terms and privacy.

110 **PUT** **/content/pages/{kind}****NOT BUILT**

Save a legal page. {kind} is terms or privacy.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
title	string	yes	
body	string	yes	HTML from the rich-text editor.
status	string	yes	published draft

RESPONSE 200 OK

The saved legal page.

111 GET /content/pages/contact

NOT BUILT

The contact page content.

RESPONSE 200 OK

```
{
  "title": "اتصل بنا",
  "introduction": "<p>نحن هنا لمساعدتك</p>",
  "supportPhone": "0931234567",
  "supportEmail": "support@mapmob.com",
  "facebookUrl": "https://facebook.com/mapmob",
  "instagramUrl": "https://instagram.com/mapmob",
  "telegramUrl": "https://t.me/mapmob",
  "whatsappUrl": "https://wa.me/963931234567"
}
```

Note. All four social URLs may be empty strings rather than null — the dashboard's form treats them as text inputs.

112 PUT /content/pages/contact

NOT BUILT

Save the contact page.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(all fields from GET, plus status)			
status	string	yes	published draft

RESPONSE 200 OK

The saved contact page.

113 GET /content/faq

NOT BUILT

The FAQ list, in display order.

RESPONSE 200 OK

```
[
  {
    "id": "q1",
    "question": "كيف أضيف منتج؟",
    "answer": "<p>صفحة التسجيل</p>"
  }
]
```

Note. Return them in the order they should appear; the dashboard numbers them by position.

114 **POST** /content/faq

NOT BUILT

Add a question.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
question	string	yes	
answer	string	yes	HTML.

RESPONSE 201 CREATED

The created question.

115 **PUT** /content/faq/{id}

NOT BUILT

Edit a question.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
question	string	yes	
answer	string	yes	HTML.

RESPONSE 200 OK

The updated question.

116 **DELETE** /content/faq/{id}

NOT BUILT

Delete a question.

RESPONSE 204 NO CONTENT

(empty body)

Settings

19 endpoints

Six settings panels: the admin's own account, the admin team, roles and permissions, which events raise an alert, accepted payment methods, and platform-wide options. Nineteen endpoints, none built. The roles panel is the one to read carefully — it drives what every admin can do.

117 GET /settings/account

NOT BUILT

The signed-in admin's own profile.

RESPONSE 200 OK

```
{
  "fullName": "Obedah",
  "email": "obedah@gmail.com",
  "roleName": "مدير عام"
}
```

Note. roleName is display text and is not editable here.

118 PUT /settings/account

NOT BUILT

Update the signed-in admin's name and email.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
fullName	string	yes	
email	string	yes	

RESPONSE 200 OK

The updated profile.

119 PUT /settings/account/password

NOT BUILT

Change the signed-in admin's password.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
currentPassword	string	yes	Must be verified server-side.
newPassword	string	yes	

RESPONSE 204 NO CONTENT

(empty body)

Note. Return 422 with a field error on currentPassword when it does not match, so the form can show it on the right input.

120 GET /settings/admins

NOT BUILT

The admin team list.

RESPONSE 200 OK

```
[
  {
    "id": "1", "fullName": "Obedah", "email": "obedah@gmail.com",
    "roleId": "1", "roleName": "مدير عام",
    "status": "active", "lastSignInOn": "2026-09-27T17:59:00Z"
  }
]
```

Note. lastSignInOn is null for an invited admin who has never signed in.

121 POST /settings/admins

NOT BUILT

Invite a new admin.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
fullName	string	yes	
email	string	yes	
roleId	string	yes	The role to assign.

RESPONSE 201 CREATED

The created admin row.

Note. The dashboard does not send a password — it expects the API to send an invitation email and create the account with status active and lastSignInOn null. Please confirm that flow.

122 GET /settings/roles

NOT BUILT

The roles list with their permission grants.

RESPONSE 200 OK

```
[
  {
    "id": "1",
    "name": "مدير عام",
    "englishName": "Super Admin",
    "description": "وصول كامل",
    "icon": "shield",
    "isFullAccess": true,
    "isActive": true,
    "grants": ["places:view", "places:add", "places:edit", "places:delete"],
    "adminCount": 2
  }
]
```

Note. grants is the key field: a flat list of "module:action" strings. The 14 modules are home, places, categories, regions, users, reviews, offers, subscriptions, payments, reports, complaints, notifications, content, system. The 4 actions are view, add, edit, delete. isFullAccess true means every permission regardless of the grants list. icon is shield | headset.

123 POST /settings/roles**NOT BUILT**

Create a role.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
description	string	yes	
isActive	boolean	yes	
grants	string[]	yes	The full list of "module:action" strings.

RESPONSE 201 CREATED

The created role.

Note. englishName, icon, isFullAccess and adminCount are not sent by the form — derive or default them.

124 PUT /settings/roles/{id}**NOT BUILT**

Update a role.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(same as POST /settings/roles)			grants replaces the whole list.

RESPONSE 200 OK

The updated role.

125 DELETE /settings/roles/{id}**NOT BUILT**

Delete a role.

RESPONSE 204 NO CONTENT

(empty body)

Note. Should be refused with 422 when admins are still assigned to it — adminCount tells the dashboard to warn first, but please enforce it server-side too.

126 GET /settings/notifications**NOT BUILT**

Which internal events raise an alert for admins.

RESPONSE 200 OK

```
[
  { "kind": "new-complaint", "isEnabled": true },
  { "kind": "place-awaiting-approval", "isEnabled": true },
  { "kind": "review-reported", "isEnabled": false },
  { "kind": "subscription-expiring", "isEnabled": true },
  { "kind": "new-payment", "isEnabled": true }
]
```

Note. Exactly these five kinds, spelled in kebab-case. Return all five rows always.

127 PUT /settings/notifications/{kind}**NOT BUILT**

Turn one alert on or off. {kind} is one of the five above.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
isEnabled	boolean	yes	

RESPONSE 200 OK

The updated alert row.

128 GET /settings/payment-methods**NOT BUILT**

The payment methods the platform accepts.

RESPONSE 200 OK

```
[
  { "id": "1", "name": "نقداً", "kind": "manual", "status": "active" }
]
```

Note. kind is manual | electronic. status is active | suspended.

129 POST /settings/payment-methods**NOT BUILT**

Add a payment method.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
name	string	yes	
kind	string	yes	manual electronic
status	string	yes	active suspended

RESPONSE 201 CREATED

The created method.

130 PUT /settings/payment-methods/{id}**NOT BUILT**

Update a payment method.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
(same as POST)			

RESPONSE 200 OK

The updated method.

Note. There is no delete endpoint — the dashboard suspends a method instead of removing it.

131 GET /settings/platform

NOT BUILT

All platform settings in one call, grouped into four blocks.

RESPONSE 200 OK

```
{
  "general": {
    "appName": "MapMob",
    "logoFileName": "logo.png",
    "supportEmail": "support@mapmob.com",
    "supportPhone": "0931234567"
  },
  "map": { "distanceUnit": "kilometer", "searchRadiusKm": 10 },
  "language": { "defaultLanguage": "ar", "detectsDeviceLanguage": true },
  "currency": { "currency": "SYP", "currencySymbol": "ل.س", "decimalPlaces": 0 }
}
```

Note. One read, four separate writes below — each panel saves on its own. Note general returns logoFileName (the name to display) while the write sends the file itself. distanceUnit is kilometer | mile; defaultLanguage is ar | en; decimalPlaces is 0, 1 or 2.

132 PUT /settings/platform/general

NOT BUILT

Save the general panel.

REQUEST BODY multipart/form-data

FIELD	TYPE	REQUIRED	NOTES
appName	string	yes	
supportEmail	string	yes	
supportPhone	string	yes	
logo	file	no	New logo. Omitted keeps the current one.

RESPONSE 200 OK

The saved general block, including the new logoFileName.

Note. Multipart because of the logo; the other three panels are JSON.

133 PUT /settings/platform/map

NOT BUILT

Save the map panel.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
distanceUnit	string	yes	kilometer mile
searchRadiusKm	number	yes	Default search radius in km.

RESPONSE 200 OK

The saved map block.

Note. searchRadiusKm stays in kilometres even when distanceUnit is mile — the unit is a display choice.

134 **PUT** **/settings/platform/language****NOT BUILT**

Save the language panel.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
defaultLanguage	string	yes	ar en
detectsDeviceLanguage	boolean	yes	

RESPONSE 200 OK

The saved language block.

135 **PUT** **/settings/platform/currency****NOT BUILT**

Save the currency panel.

REQUEST BODY application/json

FIELD	TYPE	REQUIRED	NOTES
currency	string	yes	SYP USD
currencySymbol	string	yes	Shown next to amounts.
decimalPlaces	integer	yes	0 1 2

RESPONSE 200 OK

The saved currency block.

4. Gap analysis against the current API

Checked against the Postman collection `Map Mop.postman_collection.json` and verified live against `https://api.mapmob.com.co` on 27 September 2026.

Headline numbers

The dashboard needs **135 endpoints**. **31** exist in some form (5 usable as-is, 26 needing changes). **104** return **404** today.

Everything below was verified by calling the live API with the `obedah@gmail.com` super-admin account on 27 September 2026, not just read off the Postman collection.

STATE	COUNT	MEANING
Ready to use	5	Correct path, verb and shape. All are DELETE endpoints.
Exist, need changes	26	Route works; field names, envelope or verb differ.
Not built	104	Returns 404.
Total	135	

Per-feature coverage

Where a feature shows 0 built, the whole feature is missing — the frontend for it is finished and waiting.

FEATURE	NEEDED	EXIST	MISSING	NOTES
Authentication	1	1	0	Login works; response shape differs.
Dashboard (home)	5	0	5	Whole feature missing.
Places	10	2	8	List/detail exist; create, edit, bulk actions, export missing.
Categories	6	4	2	Best-covered feature. Icon model is wrong.
Regions (governorates + areas)	11	9	2	Maps onto regions + cities. Only status toggles missing.
Users	6	2	4	List and detail exist but are thin.
Reviews	8	0	8	Whole feature missing.
Offers	10	5	5	CRUD exists; summary, pause/resume, options, export missing.
Ads	10	5	5	Same pattern as offers.
Subscriptions and plans	7	0	7	Whole feature missing.
Payments	6	0	6	Whole feature missing.
Analytics reports	5	0	5	Whole feature missing.
Push notifications	12	0	12	Whole feature missing.
Inbox	2	0	2	Whole feature missing.
Complaints	6	3	3	Exists as /admin/reports.
Content pages	11	0	11	Whole feature missing.
Settings	19	0	19	Whole feature missing.
Total	135	31	104	

Naming: the same thing under two names

Four resources already exist under a different name. These are not missing features, just a vocabulary difference. Please either rename the routes or expose aliases — whichever is cheaper — but the two sides do need to agree, because `/reports` currently means two completely different things.

DASHBOARD PATH	EXISTING API PATH	NOTE
<code>/places</code>	<code>/admin/stores</code>	Same entity. 'Place' is the word used throughout the dashboard and the app.
<code>/governorates</code>	<code>/admin/regions</code>	Top level of the location tree.
<code>/areas</code>	<code>/admin/cities</code>	Second level, nested under a governorate.
<code>/complaints</code>	<code>/admin/reports</code>	User reports against a place.
<code>/reports</code>	(nothing)	DANGER: in the dashboard <code>/reports</code> is the analytics screen (charts), NOT complaints. If the API keeps <code>/admin/reports</code> for complaints, the analytics endpoints need a different prefix.

In the Postman collection but wrong

Three problems in the saved collection itself, found while testing every request in it.

REQUEST	PROBLEM	DETAIL
store update	Sends <code>type=supermarket</code> , which returns 500	The DB enum only accepts 'store'. See issue 4.
store work hours (GET)	URL is <code>{{mapurl}}admin//stores/1/working-hours</code>	<code>/admin</code> is on the wrong side of the variable; returns 404.
region + store requests	Use <code>{{token}}</code> , working hours use <code>{{admintoken}}</code>	Three names for one token: <code>admin_token</code> , <code>token</code> , <code>admintoken</code> . Only <code>admin_token</code> is defined.

Endpoints that exist but the dashboard does not call

Five working endpoints in the collection are not used by the dashboard. They are not mistakes — they are a design difference worth a decision before anyone builds against them.

Store working hours (4 endpoints). `POST /admin/stores/{id}/working-hours`, `POST /admin/stores/{id}/working-hours/{hourId}`, `GET /admin/stores/{id}/working-hours` and `DELETE /admin/stores/{id}/working-hours/{hourId}` all work today, treating each weekday as its own record created one call at a time.

The dashboard does not work that way. Its place form edits all seven days as one grid and saves them with the rest of the place in a single submit, so this document specifies working hours as a nested `workingHours[n][...]` array inside `POST /places` and `PUT /places/{id}` (endpoints 13 and 14), and as pre-formatted display rows on `GET /places/{id}`.

Decision needed: either accept the nested array on the place write endpoints (preferred — one save, one transaction, and a half-saved week becomes impossible), or tell us to keep the four per-day endpoints and the frontend will issue up to seven extra calls per save. Either way the existing four can stay for the mobile app; they simply are not what the dashboard calls.

Store products. `POST /admin/products` exists and the dashboard never calls it directly, for the same reason: products are edited inside the place form and sent as a nested `products[n][...]` array. The one product endpoint the dashboard does need is `GET /places/{id}/offer-items` (endpoint 16), which the offer form uses to list one place's products.

EXISTING ENDPOINT	STATE	HOW THE DASHBOARD HANDLES IT INSTEAD
<code>POST /admin/stores/{id}/working-hours</code>	Works	Replaced by nested <code>workingHours[]</code> on <code>POST/PUT /places</code> .
<code>POST /admin/stores/{id}/working-hours/{hourId}</code>	Works	Same.

EXISTING ENDPOINT	STATE	HOW THE DASHBOARD HANDLES IT INSTEAD
GET /admin/stores/{id}/working-hours	Works	Returned inside GET /places/{id} instead.
DELETE /admin/stores/{id}/working-hours/{hourId}	Works	Same.
POST /admin/products	Works	Replaced by nested products[] on POST/PUT /places.

Works but undocumented

These respond correctly on the live API but are not in the collection. Please add them so the collection matches reality.

ENDPOINT	STATUS	NOTE
GET /admin/users	200	Returns users with roles. Needed by the users screen.
GET /admin/users/{id}	200	Single user.
GET /admin/products	200	All products across all stores.
GET /admin/products/{id}	200	Single product.
DELETE /admin/products/{id}	200	
GET /admin/offers/{id}	200	Collection has the list and update but not the single read.
DELETE /admin/offers/{id}	200	
POST /admin/logout	200	Revokes the current token.

Untranslated response messages

Several endpoints return the raw translation key instead of text, which means the entries are missing from the language files. The dashboard does not read the `message` field, so this breaks nothing for us — but the mobile app probably does.

Affected: `api.products_retrieved_successfully`, `api.product_created_successfully`, `api.product_deleted_successfully`, `messages.ads_retrieved`, `messages.ad_created`, `messages.ad_updated`, `messages.ad_deleted`, `messages.users_retrieved`, `api.reports_retrieved`, `api.report_retrieved`, `api.report_updated`.

Regions, cities, categories, stores, offers and working hours all return proper text, so it is only these three controllers.

Suggested build order

The conventions in section 1 come first — they affect every endpoint, and changing them later means changing everything. After that, the order below front-loads the work that unblocks the most screens.

STEP	WHAT	SIZE
0	Section 1 conventions: envelope, camelCase, zero-based pageIndex, real verbs, status words	Affects all 135
1	Fix the store `type` 500 and turn off debug mode (issue 4)	1 bug, 30 minutes
2	Align the endpoints that already exist: categories, regions/areas, users, offers, ads, complaints	26 endpoints
3	Place write endpoints + form options (issue 3) — unblocks the biggest screen	5 endpoints
4	The summary/counts endpoints across all features — small, read-only, unblocks every list header	8 endpoints
5	Status toggles, pause/resume, bulk actions, and the 6 CSV exports	20 endpoints
6	Reviews, then subscriptions and payments	21 endpoints
7	Settings (19) and content pages (11)	30 endpoints

STEP	WHAT	SIZE
8	Notifications (12), dashboard charts (5), analytics (5), inbox (2)	24 endpoints

One open question for the backend team

A few endpoints imply behaviour the dashboard cannot see, and it is worth agreeing on them before they are built rather than after:

1. Does `POST /payments` also create or extend the company's subscription? The form collects a plan, a term and a period, which suggests yes.
2. Does `POST /settings/admins` send an invitation email, and does the new admin set their own password? The dashboard never sends one.
3. Should deleting a user be a soft delete, given their reviews and favourites point at them?
4. What happens when deleting a category, governorate or area that still has children or places — refuse with 422, or cascade? The dashboard can warn the admin first either way, but it needs to know which.
5. Who computes campaign status? This document assumes the API derives active / scheduled / expired from the dates plus paused and draft flags, so the dashboard never does date maths. Please confirm.